

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky, grassy slope. In the middle ground, there are steep, rocky cliffs and green hills. The background features more distant, hazy mountain ranges under a sky filled with white and grey clouds.

# AI & Software Engineering Workshop

Week 1, Day 2: Personality and Prompts — Windows

Edik Simonian, Summer 2026



# What is a system prompt?

---

- An instruction the model sees **before every conversation**
- The user never sees it, but every reply obeys it
- It defines: who the bot is, how it talks, what it refuses to do
- **The most powerful single line in this project**

Same model + different system prompt = a completely different bot.

# Find it in the code

---

bot/config.py :

```
SYSTEM_PROMPT = (  
    "You are a knowledgeable and concise AI assistant. "  
    "Answer clearly and directly. Avoid unnecessary filler. "  
    "Keep responses appropriately brief for a chat interface."  
)
```

Live experiment: change it to *"You answer only in rhymes."*

Restart. Ask it anything.

# Build your persona

---

Pick one, or invent your own:

- 🍳 **Chef**: answers everything with a recipe angle
- 📖 **Tutor**: explains step by step, asks questions back
- 🎲 **Game master**: turns the chat into an adventure
- 🎤 **Comedian**: can't help making it funny

Write 3–5 sentences: who the bot is, how it speaks, one thing it always does, one thing it never does.

# Swap the brain

---

The model is an environment variable in `.env`:

```
AI_MODEL=gpt-oss-120b      # default  
AI_MODEL=zai-glm-4.7       # try this one too
```

- Same persona, different model: what changes?
- Speed vs depth: time the answers, compare the styles

# Pair demo

---

1. Swap seats (or share bot usernames)
2. Chat with your partner's bot for 5 minutes
3. Guess their system prompt from the replies alone
4. Feedback: one thing that works, one thing to sharpen

The faster the other person can guess your prompt, the better it is.

# Make the bot match its identity

---

Two commands still describe the *old* bot:

- `/help` : should list what **your** bot does, in its voice
- `/about` : should introduce **your** persona

Both live in `bot/handlers.py` . Edit them like you edited `/start` yesterday.

# Meet Claude Code

---

An **AI pair-programmer** that lives in your terminal.

- Reads your project: the real `bot/` files
- Writes code, runs commands, explains errors
- You ask in plain English; it edits and tests

Today you shaped your bot by hand. Now meet the tool that helps you build the next thing, *with you in charge*.



# Connect it to the workshop AI

---

From your **bot folder** ( `telegram-pythonanywhere-bot` ), run the connect target:

```
.\make.ps1 claude sk-your-key-here
```

Use the personal key handed out in class.

It checks the key, installs Claude Code if needed, points it at the workshop gateway, and launches `claude`. The connect script ships in this repo — nothing else to clone.

# Keep Claude Code connected

Want new terminals to stay connected?

```
.\make.ps1 claude sk-your-key-here -Persist
```

This saves the workshop settings in your PowerShell `$PROFILE`:

- the same personal class key
- the workshop gateway URL
- the model settings Claude Code needs

New terminal windows can run `claude` immediately.

**Shared lab machine? Skip `-Persist`.** It writes your class key into the machine's PowerShell `$PROFILE`, where the next student would inherit it — run the plain command (previous slide) once per session instead.

# Use it, but stay the boss

---

Inside `claude`, just ask in plain English:

*"Update `/about` in `bot/handlers.py` to introduce my persona, in its voice."*

- It edits the files, and can run the bot to test
- **Read every line it writes.** Can't explain it? Ask it to explain, or undo it
- *AI wrote it ≠ you understand it.* You own every line you ship

Claude Code is the assistant. **You're the engineer.**



# Today → Tomorrow

---

Today your bot has a personality, and you've met Claude Code, your pair-programmer in the terminal.

**Tomorrow:** new powers. You'll write your own slash commands, including one that calls the AI itself, with Claude Code helping you build.